Keywords: on-policy distillation • knowledge distillation • LLM training efficiency • state coverage

Rethinking On-Policy Distillation: One Training Example

A Single Query Recovers Most of Full-Dataset OPD Gain — Revealing That LLM Distillation Is Algorithm-Starved, Not Data-Starved

By Zixuan Fu, Bingxiang He, Yuxin Zuo, Haohuan Huang, Jinqian Zhang, Ruhang Xiao, Cheng Qian, Qinyu Luo, Huanang Gao, Yudong Wang, Zhiyuan Liu, Ning Ding, Chaojun Xiao (Tsinghua University et al.)

arXiv:2609.04172 • Preprint, September 2026

On-policy distillation (OPD) of large language models is widely believed to require large, diverse training datasets. This paper refutes that assumption: training OPD on a single query recovers most of the accuracy gain achieved by full-dataset training on mathematical reasoning, code, and general benchmarks across multiple model families. The finding reveals that OPD is *data-overfed but algorithm-starved* — the bottleneck lies in the optimization procedure, not the volume of training queries.

AT A GLANCE

Problem: OPD is assumed to require large diverse datasets; no theory or experiment had tested the data-minimal limit.

Why it matters: If one query suffices, OPD can be applied at very low data cost, democratizing teacher-guided alignment.

Method: Fix training steps; vary query count from 1 to the full dataset; measure accuracy and state coverage.

Result: One query achieves 71.5% state coverage and most of full-data OPD's gain; 16 queries reach 98.9% and match full data.

Evidence: Experiments across MATH-500, GSM8K, LiveCodeBench, MMLU-Pro with Qwen2.5-7B, Llama-3.1-8B, DeepSeek-R1-Distill-7B.

The Big Picture

On-policy distillation combines student-generated rollouts with dense token-level supervision from a teacher LLM. The student generates responses, the teacher evaluates them, and the student updates its parameters using those signals — with rollouts regenerated at each iteration from the updated student.

The conventional wisdom is that OPD needs a large and diverse set of training queries to cover the space of possible inputs and to produce varied rollouts for learning from. This view motivates the standard practice of assembling large question banks for distillation.

This paper asks a simple question: what happens at the data-minimal limit, i.e. when OPD trains on exactly one query? The answer, that one-shot OPD keeps improving for hundreds of steps and recovers most of full-data OPD's gain, reframes the problem of LLM alignment distillation entirely.

Why Existing Approaches Fall Short

Current OPD practice assembles datasets of thousands of training queries (e.g. 512–4096 mathematical problems), under the assumption that broad coverage is necessary for generalization. This creates two problems:

First, large datasets impose *data collection costs*: curating high-quality, diverse questions for every new target domain requires significant effort.

Second, large datasets may *mask algorithmic limitations*: if the training data is the bottleneck, then improving the algorithm yields little gain unless data is first increased. This paper reveals that the data is *not* the bottleneck, so algorithmic improvements (better teachers, better update rules, better rollout strategies) are the correct direction for future work.

Core Mechanism

The key insight is that even with a fixed single query q^*, the rollout distribution $\pi_\theta(\cdot \mid q^*)$ changes at every gradient step as θ is updated. This *dynamic rollout diversity* provides gradient signal across different student behaviours even when the input prompt is fixed.

The authors measure this via **state coverage**: the fraction of intermediate rollout states visited by a given query set that are also visited by the full dataset:

$$\text{Coverage}(Q) = \frac{|S(Q) \cap S(Q_{\text{full}})|}{|S(Q_{\text{full}})|}$$

One query achieves Coverage = 71.5% — most of the state space is reachable from a single starting point as the model evolves.

Technical Formulation

The OPD training objective is a token-level imitation loss:

$$\mathcal{L}_{\text{OPD}}(\theta) = -\mathbb{E}_{q\sim\mathcal{D}}\,\mathbb{E}_{y\sim\pi_\theta(\cdot|q)} \sum_t \log \pi_{\text{teacher}}(y_t \mid q, y_{<t})$$

where π_θ is the student, π_{teacher} is the frozen teacher, and y is a student-generated rollout.

At the single-query limit with query q^*, the gradient at step k is:

$$\nabla_\theta \mathcal{L} = -\mathbb{E}_{y\sim\pi_\theta(\cdot|q^*)} \sum_t \nabla_\theta \log \pi_\theta(y_t \mid q^*, y_{<t}) \cdot \log \pi_{\text{teacher}}(y_t \mid q^*, y_{<t})$$

Despite the fixed q^*, the expectation is over the current student rollout distribution, which shifts every step — providing effective gradient diversity.

4.1 Coverage Saturation

Empirically, coverage $C(N)$ as a function of query count N follows an approximate power law:

$$C(N) \approx 1 - \alpha N^{-\beta}$$

with fitted constants α, β. Empirically, $C(1) \approx 0.715$ and $C(16) \approx 0.989$.

Learning or Inference Procedure

The authors fix the total training budget (number of gradient steps) and vary only the number of unique training queries. For each query count $N \in \{1, 4, 16, 64, 512\}$:

1. Uniformly sample (or select by diversity) N queries from the training pool.
2. Run OPD for the fixed number of steps on those N queries.
3. Evaluate on held-out benchmarks (MATH-500, GSM8K, LiveCodeBench, MMLU-Pro).
4. Measure state coverage by intersecting rollout state sets.

Query selection at $N > 1$ uses a semantic diversity criterion (maximum marginal relevance on embedding space) to maximise coverage gain.

What the Guarantee Says

The paper does not prove formal guarantees but establishes an empirical regularity: OPD accuracy as a function of query count follows the same coverage power law. Because accuracy and coverage co-move, maximising *semantic diversity* of a small query set — rather than maximising dataset size — is the correct strategy for efficient OPD.

The corollary is that OPD improvements will be unlocked by better algorithms (improved rollout strategies, better acceptance rules, curriculum over query difficulty) rather than by larger datasets.

Experimental Findings

Accuracy vs. query count (Qwen2.5-7B, MATH-500):

Queries	MATH-500	Coverage
1	68.4%	71.5%
4	70.1%	89.2%
16	71.8%	98.9%
512	72.3%	100%

The accuracy gain from 1 query to 512 queries is only 3.9 percentage points, while the gain from the pre-OPD baseline to 1-query OPD is substantially larger — confirming that most of the alignment benefit comes from the OPD mechanism itself, not the data diversity.

Results are consistent across Llama-3.1-8B and DeepSeek-R1-Distill-7B student models, and across GSM8K, LiveCodeBench, and MMLU-Pro benchmarks.

Ablations and Interpretation

Random vs. semantic query selection: At $N = 16$, randomly sampled queries underperform semantically diverse queries by 1.7 percentage points (MATH-500). Coverage is the active ingredient.

Step budget vs. data size: One-shot OPD requires $\sim 2\times$ more gradient steps to match full-data OPD accuracy at $N = 16$, but converges to the same value — showing the algorithm can compensate for data sparsity given sufficient compute.

Teacher quality: Weaker teachers (same-size models) yield diminished but still positive one-shot gains. The mechanism does not require a very large teacher.

Alignment pace: Plotting validation accuracy vs. steps for $N = 1$ and $N = 512$, both curves plateau at the same rate — confirming the algorithm bottleneck hypothesis.

Reference

Zixuan Fu, Bingxiang He, Yuxin Zuo et al. **Rethinking On-Policy Distillation of Large Language Models II: One Training Example.** arXiv:2609.04172, September 2026. https://arxiv.org/abs/2609.04172

Keywords: unified multimodal models • visual understanding • image generation • task-decoupled architecture

Uncovering Understanding–Generation Synergy in Unified Multimodal Models

Joint Training Helps Both Tasks — But Only When Visual Computation Is Architecturally Decoupled by Objective

By Penghao Wu, Haiwen Diao, Weichen Fan, Lewei Lu, Dahua Lin, Ziwei Liu (NTU S-Lab; SenseTime Research)

arXiv:2609.01607 • Preprint, September 2026

Unified multimodal models (UMMs) jointly perform visual understanding and image generation within a single model. Do the two objectives help each other, compete, or merely coexist? This paper answers through a three-level investigation — representation, task, and system — in a controlled setting without pretrained vision priors. The verdict: synergy is real at the representation level, but forces both objectives through the same computation path and one will dominate. A task-decoupled architecture that specialises conflicting visual computation while preserving shared semantic interaction avoids this failure mode and yields consistent gains on both objectives simultaneously.

AT A GLANCE

Problem: Does joint understanding/generation training help, hurt, or have no effect when both objectives share the same model?

Why it matters: Unified models promise efficiency, but training instability and objective conflict limit their practical quality.

Method: Three-level analysis (representation, task, system) with task-decoupled architectural fix.

Result: Decoupled architecture improves understanding by 3.2% and generation FID by 21% vs. naive joint baseline.

Evidence: Controlled experiments at 300M–3B parameters; VQA, image captioning, and text-to-image generation benchmarks.

The Big Picture

Large vision-language models have converged on two paradigms: dedicated understanding models (e.g. LLaVA, InternVL) and dedicated generation models (e.g. Stable Diffusion, DALL-E). Unifying both in a single model is appealing: shared representations could reinforce each other, reduce model count in deployment, and enable joint reasoning over perception and synthesis.

Yet practitioners have observed that naive joint training frequently degrades one or both tasks compared to separate specialist models. The reasons are poorly understood. This paper systematically characterises the conflict and proposes a structural solution.

Why Existing Approaches Fall Short

Existing unified models (e.g. Emu, Show-o, Janus) attempt joint training but either (a) use a pretrained vision encoder that introduces its own inductive bias and masks the training interaction, or (b) report mixed results with no theoretical explanation. Three failure modes emerge:

Objective dominance. When both tasks share the same visual encoding layers, the task with stronger or more numerous gradients suppresses the other — whichever has lower loss tends to pull parameters away from the other objective's optimal region.

Representation conflict. Optimal representations for understanding encode semantic categories and attributes; optimal representations for generation encode low-level textures and pixel statistics. Forcing both through the same encoder causes a compromise that is suboptimal for both.

Confounded benchmarks. Papers using pretrained encoders cannot isolate the effect of joint training from the encoder's pretrained features.

Core Mechanism

The paper identifies that conflict arises in *early visual encoding components* (tokeniser, first few attention layers) but not in *semantic interaction layers* (cross-attention, late transformer blocks).

This motivates the task-decoupled architecture:

- **Specialised early encoders:** separate but lightweight visual tokeniser branches for understanding (Enc_u) and generation (Enc_g).
- **Shared semantic backbone:** a single transformer stack processes fused semantic representations from both branches.

Decoupling allows each encoder to optimise its own visual features while the shared backbone captures cross-modal semantics that benefit both tasks.

Technical Formulation

4.1 Joint Training Objective

The standard (conflict-prone) joint objective is:

$$\mathcal{L}_{\text{joint}} = \lambda_u \mathcal{L}_{\text{understanding}} + \lambda_g \mathcal{L}_{\text{generation}}$$

where $\mathcal{L}_u$ is cross-entropy over VQA/captioning tokens and $\mathcal{L}_g$ is a denoising diffusion loss or autoregressive token loss.

4.2 Dominance Score

To quantify asymmetric degradation, the dominance score D is:

$$D = \frac{\text{Acc}_u^{\text{joint}} / \text{Acc}_u^{\text{separate}}}{\text{Acc}_g^{\text{joint}} / \text{Acc}_g^{\text{separate}}}$$

$D \gg 1$: understanding dominates. $D \ll 1$: generation dominates. Naive joint training produces $D \neq 1$, confirming dominance. The task-decoupled model produces $D \approx 1$.

4.3 Decoupled Forward Pass

Given image x and text tokens t:

$$z_u = \text{Enc}_u(x) \tag{1}$$
$$z_g = \text{Enc}_g(x) \tag{2}$$
$$z_{\text{shared}} = \text{TransformerShared}(z_u, z_g, t) \tag{3}$$
$$\mathcal{L}_{\text{decoupled}} = \mathcal{L}_u(z_{\text{shared}}) + \mathcal{L}_g(z_{\text{shared}}) \tag{4}$$

Both encoders see the shared backbone's gradients, enabling beneficial cross-task signal, while their independent parameters specialise without conflict.

4.4 Representation-Level Synergy (CKA)

Synergy is measured via Centred Kernel Alignment (CKA) between understanding-task-optimal and generation-task-optimal representations. After decoupled training, CKA of the encoder outputs is lower (more specialised), but CKA of the semantic backbone outputs is higher (more shared) — confirming specialisation at the encoder and consolidation at the semantic level.

Learning or Inference Procedure

Training follows standard autoregressive pretraining on interleaved image-text data, with the following modification:

1. Separate learnable visual tokenisers are initialised from a shared initialisation but trained independently.
2. Gradients from $\mathcal{L}_u$ update Enc_u and the shared backbone; gradients from $\mathcal{L}_g$ update Enc_g and the shared backbone.
3. The shared backbone receives signal from both tasks at every step.

At inference, only the relevant encoder branch is activated: understanding queries use Enc_u; generation queries use Enc_g.

What the Guarantee Says

The paper proves that under mild smoothness conditions on the loss landscape, the decoupled architecture has a strictly smaller gradient conflict norm than the fully shared architecture:

$$\|\nabla_\theta \mathcal{L}_u + \nabla_\theta \mathcal{L}_g\|^2_{\text{decoupled}} < \|\nabla_\theta \mathcal{L}_u + \nabla_\theta \mathcal{L}_g\|^2_{\text{shared}}$$

whenever the encoder Jacobians for understanding and generation are non-parallel — which holds whenever the two tasks require different visual features, a natural assumption.

Experimental Findings

Understanding benchmarks (VQA, image classification): The task-decoupled model achieves 3.2% average relative improvement over the best single-task baseline and 5.8% over the naive joint model.

Generation benchmarks (FID on COCO, 256×256):

Model	FID↓	Params
Generation-only specialist	7.4	1B
Naive joint UMM	10.3	1B
Task-decoupled (ours)	**8.1**	1.1B

Parameter overhead: The two specialised encoders add 12% extra parameters vs. the fully shared model, but outperform it on both tasks simultaneously.

Ablations and Interpretation

Which layers to specialise? Specialising only the visual tokeniser (earliest layers) provides the largest gain. Specialising all layers including late transformer blocks hurts by breaking shared semantic grounding.

Loss weight sensitivity (λ_u, λ_g): The naive joint model is highly sensitive to loss weighting: accuracy fluctuates by up to 8% as λ_u/λ_g varies from 0.5 to 2.0. The decoupled model is robust across this range.

Scaling (300M, 1B, 3B): The performance gains persist at all three scales, suggesting the conflict is a structural, not a capacity, issue.

CKA trajectories: During training, understanding and generation encoder representations diverge (lower cross-task CKA), while semantic backbone representations converge (higher cross-task CKA) — matching the specialisation-then-fusion theory.

Reference

Penghao Wu, Haiwen Diao, Weichen Fan, Lewei Lu, Dahua Lin, and Ziwei Liu. **Uncovering Understanding-**

Generation Synergy in Native Unified Multimodal Models: From Representation, Task to System. arXiv:2609.01607, September 2026. https://arxiv.org/abs/2609.01607

Keywords: pixel-space diffusion • image generation • sparse attention • global refinement

Advanced Pixel Diffusion with Guided Sparse Global Refinement (PixSGR)

A Low-Channel Bottleneck Plus Sparse Cross-Patch Attention Brings Pixel Diffusion to Latent Diffusion Quality at Substantially Lower Cost

By Weiyi You, Jinhua Zhang, Xingyu Zhou, Wei Long, Junyu Lou, Shuhang Gu

arXiv:2609.00798 • Preprint, September 2026

Pixel-space diffusion offers higher fidelity and finer-grained control than latent diffusion, but its computational cost has kept it behind in practice. PixSGR resolves the core bottleneck with two innovations: a supervised low-channel bottleneck that captures the low-dimensional manifold of natural images before diffusion begins, and a sparse global refinement step that restores structural coherence across patch boundaries at sub-quadratic cost. On ImageNet 256×256, PixSGR matches latent diffusion quality (FID 3.41 vs. 3.60 for LDM-4) while operating entirely in pixel space.

AT A GLANCE

Problem: Pixel-space diffusion is too computationally expensive and produces structural discontinuities at patch boundaries.

Why it matters: Pixel-space models avoid latent encoder artifacts, enabling pixel-perfect fine-grained control.

Method: Supervised low-channel bottleneck + patch-level denoising + sparse global cross-patch refinement.

Result: FID 3.41 on ImageNet 256×256, matching LDM-4 with 40% fewer FLOPs than dense pixel transformer baselines.

Evidence: Ablations confirming sparse global refinement (FID 3.41 vs. 6.82 without it) and supervised bottleneck value.

The Big Picture

Latent diffusion models (LDMs) dominate image generation by compressing images into a learned latent space before applying the diffusion process. This reduces computation dramatically but introduces a structural limitation: the quality ceiling is bounded by the encoder-decoder and artifacts from the latent representation propagate to the output.

Pixel-space diffusion avoids these artifacts by modelling the image directly, but faces two challenges. First, a 256×256 RGB image has 196,608 dimensions — three orders of magnitude more than a typical latent code. Second, existing pixel diffusion approaches either use large patches (sacrificing fine detail) or process patches independently (losing structural coherence across boundaries).

PixSGR addresses both challenges with a staged architecture that keeps the effective dimensionality low while restoring global structure.

Why Existing Approaches Fall Short

Dense pixel transformers apply attention over all pixel tokens jointly, achieving global coherence at the cost of $O(N^2)$ attention where $N \approx 200\text{K}$ — computationally prohibitive at training scale.

Patch-level diffusion (e.g. DiT in pixel space) divides the image into non-overlapping patches and denoises each independently. This achieves linear cost but produces boundary artifacts: adjacent patches generate independently without seeing each other's context, leading to discontinuities in colour, texture, and large-scale structure.

Hierarchical pixel diffusion uses coarse-to-fine generation but still confines refinement within each patch, missing long-range interactions needed for global structural consistency.

Core Mechanism

PixSGR operates in three stages:

Stage 1: Supervised Low-Channel Bottleneck. A supervised autoencoder compresses the image from $C = 3$ channels to $C' = 4$ channels at the original spatial resolution. This is not a latent encoder: spatial resolution is preserved. The bottleneck captures the low-dimensional manifold of natural images in colour-space without spatial downsampling.

Stage 2: Patch-Level Diffusion. The diffusion process operates on non-overlapping patches of the bottleneck representation. A patch-level transformer denoiser processes patches in parallel — linear in the number of patches.

Stage 3: Sparse Global Refinement. After patch-level generation, a sparse set of refinement tokens is positioned at patch boundaries and a random interior subset. Each refinement token attends globally to all patch tokens via sparse attention, restoring cross-patch coherence.

Technical Formulation

4.1 Patch-Level Denoising Objective

The denoising loss over patches is the standard DDPM objective:

$$\mathcal{L}_{\text{patch}} = \mathbb{E}_{t,x_0,\varepsilon}\|\varepsilon - \varepsilon_\theta(x_t^{\text{patch}}, t)\|^2$$

where x_t^{patch} is the noisy patch at timestep t.

4.2 Sparse Global Attention

Let $P = \{p_1, \ldots, p_N\}$ be the full set of patch token positions and $R = \{r_1, \ldots, r_M\}$ with $M \ll N$ be the sparse refinement positions (straddling patch boundaries plus a random interior subset). The refinement attention weight for refinement token r_i over patch token p_j is:

$$A(r_i, p_j) = \frac{\exp\left(q(r_i)^\top k(p_j)/\sqrt{d}\right)}{Z_i} \quad \text{for } j \in \mathcal{N}_{\text{local}}(r_i) \cup \mathcal{N}_{\text{rand}}(r_i)$$

where $\mathcal{N}_{\text{local}}$ is a fixed local neighbourhood and $\mathcal{N}_{\text{rand}}$ is a randomly sampled global subset. Total attention cost is $O(N \log N)$ per refinement step.

4.3 Guided Conditioning

The refinement token hidden states are conditioned on the supervised bottleneck representation at the corresponding spatial positions:

$$h_{\text{refine}}(r_i) = \text{Attention}(r_i, P) + \gamma \cdot \text{Proj}(\text{bottleneck}(r_i))$$

where γ is a learnable scalar. This structural anchoring prevents the refinement step from drifting from the coarse image structure.

Learning or Inference Procedure

Training proceeds in two phases:

1. **Bottleneck pre-training.** Train the low-channel autoencoder with a reconstruction loss on ImageNet. The encoder produces the 4-channel spatial bottleneck; the decoder reconstructs the original image from patch-level outputs.
2. **Joint diffusion training.** Train the patch-level denoiser and the sparse global refinement module jointly on the bottleneck representation, using the DDPM objective.

At inference, the model runs the diffusion reverse process patch-by-patch, then applies one sparse global refinement pass over the fully denoised bottleneck before decoding to pixels.

What the Guarantee Says

The paper establishes that the sparse global attention achieves an approximation error bound:

$$\|A_{\text{sparse}} - A_{\text{dense}}\|_F \leq \epsilon_{\text{local}} + \epsilon_{\text{rand}}$$

where ϵ_{local} decreases with the local neighbourhood radius and ϵ_{rand} decreases with the random sampling budget. For the default configuration (16-token local neighbourhood, 5% random sample), $\epsilon_{\text{rand}} + \epsilon_{\text{local}}$ is bounded under standard assumptions on the attention score distribution.

Experimental Findings

Class-conditional generation, ImageNet 256×256:

Model	FID↓	Params
LDM-4 (latent diffusion)	3.60	400M
DiT-XL/2 (latent)	2.27	675M
Pixel DiT (dense)	8.90	600M
PixSGR (ours)	**3.41**	550M

PixSGR matches LDM-4 quality in pixel space with 8.5% fewer parameters than the dense pixel diffusion baseline, and reduces FLOPs per denoising step by 40%.

On FFHQ 256×256 (unconditional), PixSGR achieves FID 4.2 vs. 5.9 for the dense pixel baseline, with similar parameter counts.

Ablations and Interpretation

Without sparse global refinement: FID degrades from 3.41 to 6.82 on ImageNet — the largest single ablation factor, confirming that cross-patch coherence is the critical missing ingredient in prior pixel diffusion.

Without supervised bottleneck: Convergence slows substantially and final FID increases to 5.10; the bottleneck reduces the effective dimensionality of the diffusion task from the start.

Refinement token density: FID improves as the sparse refinement fraction increases from 5% to 15% of total patches, then plateaus. The default of 10% gives the best efficiency-quality trade-off.

Guided vs. unguided refinement: Adding the bottleneck conditioning term (guided) reduces FID by 0.8 points over unguided refinement, demonstrating that structural anchoring is beneficial.

Local neighbourhood radius: Increasing the local neighbourhood from 4 to 16 tokens reduces FID by 0.4 points; beyond 16, gains are marginal and cost increases.

Reference

Weiyi You, Jinhua Zhang, Xingyu Zhou, Wei Long, Junyu Lou, and Shuhang Gu. **Advanced Pixel Diffusion Model with Guided Sparse Global Refinement.** arXiv:2609.00798, September 2026. https://arxiv.org/abs/2609.00798

Keywords: vision-language-action models • embodied AI • robot manipulation • skill-based planning

EmbodiedSkills: Orchestrating, Training, and Deploying VLA Agents

A Unified Skill Interface with Precondition Checking and Postcondition Verification Lifts Robot Manipulation to 86% Success

By Wei Wang, Wenqiao Zhang, Yutong Lin, et al. (17 authors; Zhejiang University et al.)

arXiv:2609.01281 • Preprint, September 2026

Vision-Language-Action (VLA) models map visual observations and language instructions to robot actions end-to-end, but they struggle with long-horizon tasks because early errors cascade without a recovery mechanism. EmbodiedSkills wraps VLA policies in an executable-skill abstraction that checks prerequisites before execution and verifies outcomes afterward. Task-adapted low-level VLA policies within this framework achieve 86.20% average success across 50 RoboTwin 2.0 tasks and 97.40% across four LIBERO suites — far above frozen-policy baselines.

AT A GLANCE

Problem: VLA models fail on long-horizon tasks because they have no mechanism to detect or recover from execution errors.

Why it matters: Real-world robot deployment requires reliable multi-step manipulation; a single failure can ruin a long task.

Method: Executable-skill interface with learned precondition and postcondition classifiers; modular VLA policy adaptation.

Result: 86.20% average success (RoboTwin 2.0), 97.40% (LIBERO); 62% reduction in cascading failures on 6+ skill tasks.

Evidence: Comparison vs. frozen OpenVLA and task-adapted VLA without skill structure across 54 robot manipulation tasks.

The Big Picture

VLA models have demonstrated impressive generalisation on short-horizon manipulation tasks: given an image observation and a language description, they generate motor commands that successfully complete simple pick-and-place or push tasks. However, real-world applications require long-horizon plans: setting a table, assembling a product, or restocking a shelf involves sequences of 6–10 or more distinct sub-tasks.

In long-horizon settings, VLA models fail because they lack two critical properties:

- **Error detection:** the model has no way to know whether a sub-task succeeded or failed before moving to the next.
- **Recovery:** when a sub-task fails, the model cannot re-try or select an alternative action sequence.

EmbodiedSkills introduces both properties by structuring VLA execution around *executable skills* — a design pattern borrowed from classical task-and-motion planning but implemented entirely with learned neural components.

Why Existing Approaches Fall Short

Monolithic VLA policies (e.g. OpenVLA, RT-2) are trained end-to-end and have no explicit sub-task structure. They may hallucinate task completion or proceed past failures.

Hierarchical RL approaches train separate high-level and low-level policies, but the high-level policy must be retrained for each new task, and low-level sub-policies typically cannot be reused across hierarchies.

Classical task planners (e.g. PDDL-based) offer structured execution with precondition/postcondition semantics but require hand-coded predicates and cannot handle the visual complexity of real-world robot environments.

EmbodiedSkills occupies the gap: learned precondition/postcondition classifiers from visual observations, structured execution via a fixed skill interface, and task-adapted VLA policies as low-level executors.

Core Mechanism

A skill s is a tuple:

$$s = (\texttt{name},\ \texttt{pre}(o),\ \pi_s(o, g),\ \texttt{post}(o))$$

where:

- $\texttt{pre}(o) \to \{0, 1\}$: checks whether the robot is in a valid state to begin executing s.
- $\pi_s(o, g) \to a$: the low-level VLA policy that generates actions to achieve sub-goal g from observation o.
- $\texttt{post}(o) \to \{0, 1\}$: checks whether skill s completed successfully.

The agent loop selects skills via a vision-language model planner, checks preconditions, executes until postcondition is satisfied or a timeout is reached, and triggers recovery or replanning on failure.

Technical Formulation

4.1 High-Level Planning

The high-level planner is a VLM queried at each step as:

$$s_t, a_t = \text{VLM}(G, o_t, \mathcal{S}) \tag{1}$$

where G is the task goal, o_t is the current observation, and $\mathcal{S}$ is the skill library. The VLM reasons in natural language via chain-of-thought before selecting the next skill s_t and its arguments a_t.

4.2 Precondition and Postcondition Classifiers

Pre- and postconditions are binary classifiers trained on annotated demonstrations:

$$f_{\text{pre}}(o;\, \theta_{\text{pre}}) \to P(\texttt{pre satisfied}) \tag{2}$$
$$f_{\text{post}}(o;\, \theta_{\text{post}}) \to P(\texttt{post satisfied}) \tag{3}$$

with positive/negative labels derived from states before and after successful vs. failed skill executions.

4.3 Task-Adapted Policy Training

Each skill's low-level policy π_s is trained with a combination of behavioural cloning and proximal policy optimisation:

$$\mathcal{L}_s(\theta) = \mathcal{L}_{\text{BC}}(\theta) + \lambda\, \mathcal{L}_{\text{PPO}}(\theta, R_s)$$

where R_s is a dense reward derived from the postcondition classifier:

$$R_s(o_t) = f_{\text{post}}(o_t;\, \theta_{\text{post}})$$

This reward is dense — it provides a training signal at every step proportional to how close the current state is to satisfying the postcondition — avoiding the sparse reward problem common in robot RL.

4.4 Recovery Policy

Failure modes are classified by the postcondition checker; each failure mode has an associated recovery policy r selected from a predefined recovery library:

$$r_t = \text{RecoveryPolicy}(o_t, s_t, \text{failure\_mode})$$

Learning or Inference Procedure

EmbodiedSkills is trained in three stages:

1. **Precondition/postcondition learning.** Classifiers are trained on annotated state transitions from expert demonstrations.
2. **Low-level VLA adaptation.** Each skill policy is fine-tuned from a pretrained VLA (OpenVLA) using BC + PPO with the postcondition reward. Adaptation is independent per skill.
3. **High-level planner.** A pretrained VLM (7B or 72B) is prompted with the skill library and task specification; no VLM fine-tuning is required.

At inference, the agent loop runs: plan → check precondition → execute → verify postcondition → recover or continue.

What the Guarantee Says

The paper proves that under the learned classifier regime, the expected task completion rate $P(\text{task})$ satisfies:

$$P(\text{task}) \geq \prod_{i=1}^{K} P(\text{success}_i) \cdot (1 - P(\text{cascade}))$$

where K is the number of skills, $P(\text{success}_i)$ is the per-skill success rate, and $P(\text{cascade})$ is the probability of a cascade failure that the recovery policy cannot resolve. Adding postcondition verification suppresses $P(\text{cascade})$ by catching failures before the next skill begins.

Experimental Findings

Method	RoboTwin 2.0	LIBERO
Frozen VLA (OpenVLA)	51.4%	78.6%
Task-adapted (no framework)	71.2%	89.1%
EmbodiedSkills	**86.2%**	**97.4%**

Long-horizon tasks (8+ skills): EmbodiedSkills achieves 74.3% completion vs. 31.2% for the task-adapted baseline without skill structure, a 2.4× improvement attributable to verification-driven recovery.

Ablations and Interpretation

Without postcondition verification: Success drops from 86.2% to 61.4% on RoboTwin 2.0 — the largest single ablation contributor. Cascading failures account for the majority of the 24.8-point gap.

Without precondition checking: Success drops to 74.8%, showing that preventing skill execution from an invalid start state avoids a different class of compounding errors.

VLM planner size: Using a 7B planner vs. a 72B planner reduces success by 4.1% on RoboTwin 2.0. The structured skill interface compensates for planner imperfection: the planner only needs to choose the right skill, not the right low-level action.

Fixed vs. adaptive skill library: A fixed library of 28 primitive skills matches a dynamically grown library for the tested tasks, suggesting that a compact skill vocabulary covers the space of practical manipulation scenarios.

Dense reward (postcondition) vs. sparse reward:

Using the postcondition classifier as a dense reward (vs. sparse 0/1 completion reward) increases average per-skill success rate by 11.4 points.

Reference

Wei Wang, Wenqiao Zhang, Yutong Lin et al. **EmbodiedSkills: A Unified Framework for Orchestrating, Training, and Deploying VLA Agents.** arXiv:2609.01281, September 2026. https://arxiv.org/abs/2609.01281

Keywords: speculative decoding • factuality • hallucination mitigation • inference efficiency

SFAD: Speculative Factuality-Aware Decoding

A DPO-Trained Draft Model Plus Epistemic Friction Enforces Contextual Faithfulness at Speculative Decoding Speeds

By Guanqiao Chen, Di Wang, Lijie Hu (MBZUAI et al.)

arXiv:2609.00796 • Preprint, September 2026

Contextual faithfulness — the property that a language model's output is consistent with a provided source document — is critical for retrieval-augmented generation, summarisation, and question answering. Existing mitigation strategies either double inference cost (contrastive decoding) or require expensive post-training (RL alignment). SFAD achieves a 38% relative reduction in hallucination rate and an 11.6-point FactScore gain over greedy decoding, at throughput matching standard speculative decoding, by training a factuality-specialised draft model with fine-grained preference data and enforcing faithfulness online via an Epistemic Friction detector.

AT A GLANCE

Problem: LLMs hallucinate even when source documents are provided; existing remedies sacrifice inference speed.

Why it matters: RAG and summarisation systems used in production cannot tolerate either hallucinations or latency overhead.

Method: DPO-trained context-faithful draft model (ConFide data) + Epistemic Friction detector; no change to verifier LLM or workflow.

Result: FactScore 72.8% vs. 61.2% greedy; 88 tok/s vs. 91 tok/s speculative decoding (no throughput penalty).

Evidence: RAGTruth, TruthfulQA, SummEval benchmarks; ablations isolating ConFide DPO and Epistemic Friction contributions.

The Big Picture

Retrieval-augmented generation (RAG) grounds language model outputs in retrieved source documents. Despite the presence of relevant documents in context, large language models still hallucinate — generating plausible-sounding but contextually unfaithful text that contradicts or is unsupported by the source.

The challenge is threefold. First, hallucination occurs not because the model lacks information but because the decoding process does not explicitly enforce consistency with the context. Second, the most effective existing methods (contrastive decoding) require two forward passes, doubling inference cost. Third, post-training alignment via RL requires significant compute and careful reward design.

SFAD is the first approach to embed factuality enforcement inside speculative decoding, achieving faithfulness gains with zero throughput overhead.

Why Existing Approaches Fall Short

Contrastive decoding compares model outputs under different prompts (faithful vs. non-faithful) and selects tokens with higher probability under the faithful prompt. It requires two forward passes per token, doubling inference cost — impractical in production.

RL post-training alignment trains the model to prefer faithful outputs via reward models, but requires thousands of feedback examples and careful reward shaping. The resulting model still hallucinate under distribution shift.

Standard speculative decoding uses a small draft model to propose multiple tokens for a large verifier LLM to accept/reject in parallel. This achieves 2–3× speedup but makes no claim about the faithfulness of accepted tokens.

Core Mechanism

SFAD modifies the draft model in speculative decoding to be *context-faithful*, so that the token proposals it generates are more likely to be grounded in the source document. The verifier LLM then accepts these proposals with high probability — because they are both fluent (the draft is a capable LLM) and faithful (the draft is DPO-trained on faithfulness preferences). No modification to the verifier or the acceptance criterion is required.

An additional online component, Epistemic Friction, detects tokens where the draft's proposal diverges from the verifier's preference in a context-relevant direction, and steers the draft before it proposes — further increasing acceptance rate and faithfulness.

Technical Formulation

4.1 ConFide Dataset

ConFide is a preference dataset of (context, chosen output, rejected output) triples, constructed from model outputs on knowledge-intensive prompts via *atomic perturbations*:

- **Entity swap**: replace entity names with plausible alternatives from the knowledge base.
- **Relation inversion**: change predicate direction (e.g. "A was founded by B" → "B was founded by A").

- **Quantifier change**: replace "all" with "some", exact numbers with nearby values.
- **Temporal shift**: shift dates or tenses.

Original outputs (faithful to source) are chosen; perturbed outputs are rejected. This produces fine-grained atomic contrasts that teach the draft model exactly which token sequences are unfaithful.

4.2 DPO Training

The draft model π_θ is fine-tuned on ConFide with Direct Preference Optimisation:

$$\mathcal{L}_{\text{DPO}} = -\mathbb{E}_{(x,y_+,y_-)}\big[\log\sigma\big(\beta(\log\pi_\theta(y_+|x) - \log\pi_{\text{ref}}(y_+|x)) - \beta(\log\pi_\theta(y_-|x) - \log\pi_{\text{ref}}(y_-|x))\big)\big] \quad (1)$$

where y_+ is the faithful (chosen) output, y_- is the perturbed (rejected) output, and π_{ref} is the pre-DPO draft.

4.3 Epistemic Friction

At each proposed token y_t, the Epistemic Friction score is:

$$\text{EF}(t) = \text{KL}(\pi_{\text{draft}}(\cdot|x, y_{<t}) \parallel \pi_{\text{verifier}}(\cdot|x, y_{<t}))\cdot w_{\text{specialist}}(t)$$

where $w_{\text{specialist}}(t)$ is the verifier's mean attention weight to the source document at position t, measuring how context-relevant the current generation position is.

High $\text{EF}(t)$ flags a likely hallucination site. The draft's sampling temperature is increased at these positions to diversify proposals away from the unfaithful mode.

4.4 Speculative Acceptance

The standard speculative decoding acceptance criterion is unchanged:

$$P(\text{accept } y_t) = \min\left(1,\ \frac{\pi_{\text{verifier}}(y_t|x, y_{<t})}{\pi_{\text{draft}}(y_t|x, y_{<t})}\right)$$

Because the DPO draft assigns higher probability to faithful tokens, the verifier's acceptance rate increases for faithful proposals and decreases for unfaithful ones — without any change to the acceptance formula.

Learning or Inference Procedure

Training:

1. Sample model outputs on knowledge-intensive prompts.
2. Apply atomic perturbations to construct ConFide preference pairs.
3. Fine-tune a small draft LLM (3–7B parameters) with DPO on ConFide.

Inference:

1. Draft model proposes $k = 4$ tokens per speculative step.
2. Compute EF for each proposed token; re-sample tokens above the EF threshold.
3. Verifier LLM processes the k (steered) tokens in parallel and accepts/rejects via the standard criterion.
4. Accepted tokens are output; rejected tokens restart from the last accepted position.

The EF computation adds a single matrix product over the verifier's cached attention weights — negligible overhead.

What the Guarantee Says

The paper proves that under DPO training, the faithful token acceptance rate improves by at least:

$$\Delta_{\text{accept}} \geq \frac{\beta}{\mathcal{Z}}\big(\mathbb{E}[\log\pi_{\text{DPO}}(y_+)] - \mathbb{E}[\log\pi_{\text{ref}}(y_+)]\big)$$

which is strictly positive whenever DPO training increases the faithful log-probability above the reference model — a necessary condition for successful DPO training. The bound is tight for token-level independent acceptance.

Experimental Findings

Main results across benchmarks:

Method	FactScore↑	Throughput
Greedy (verifier only)	61.2%	38 tok/s
Contrastive decoding	68.4%	19 tok/s
Standard speculative decoding	61.3%	91 tok/s
SFAD (ours)	**72.8%**	**88 tok/s**

SFAD improves FactScore by 11.6 points over greedy and 4.4 points over contrastive decoding, while matching standard speculative decoding throughput. On RAGTruth specifically, hallucination rate drops from 28.3% to 17.5% (38% relative reduction).

Ablations and Interpretation

Without ConFide DPO: FactScore drops by 6.8 points — the draft model fine-tuning is the primary source of faithfulness gain.

Without Epistemic Friction: FactScore drops by 2.1 points, showing that online steering provides a complementary, additive faithfulness layer.

Atomic vs. coarse perturbations in ConFide: Fine-grained atomic perturbations outperform sentence-level contrastive pairs by 3.4 FactScore points, validating that granular contrast is essential for teaching the draft model about specific faithfulness failures.

Specialist weight $w_{\text{specialist}}$: Attention-based specialist

weighting outperforms uniform weighting by 1.8 FactScore points. The model is most useful at context-relevant positions, not uniformly.

EF threshold sensitivity: FactScore is robust to the EF threshold in the range $[0.3, 0.7]$; below 0.3 too many tokens are resampled (hurting fluency); above 0.7 too few hallucinations are caught.

Reference

Guanqiao Chen, Di Wang, and Lijie Hu. **SFAD: Speculative Factuality-Aware Decoding.** arXiv:2609.00796, September 2026. https://arxiv.org/abs/2609.00796